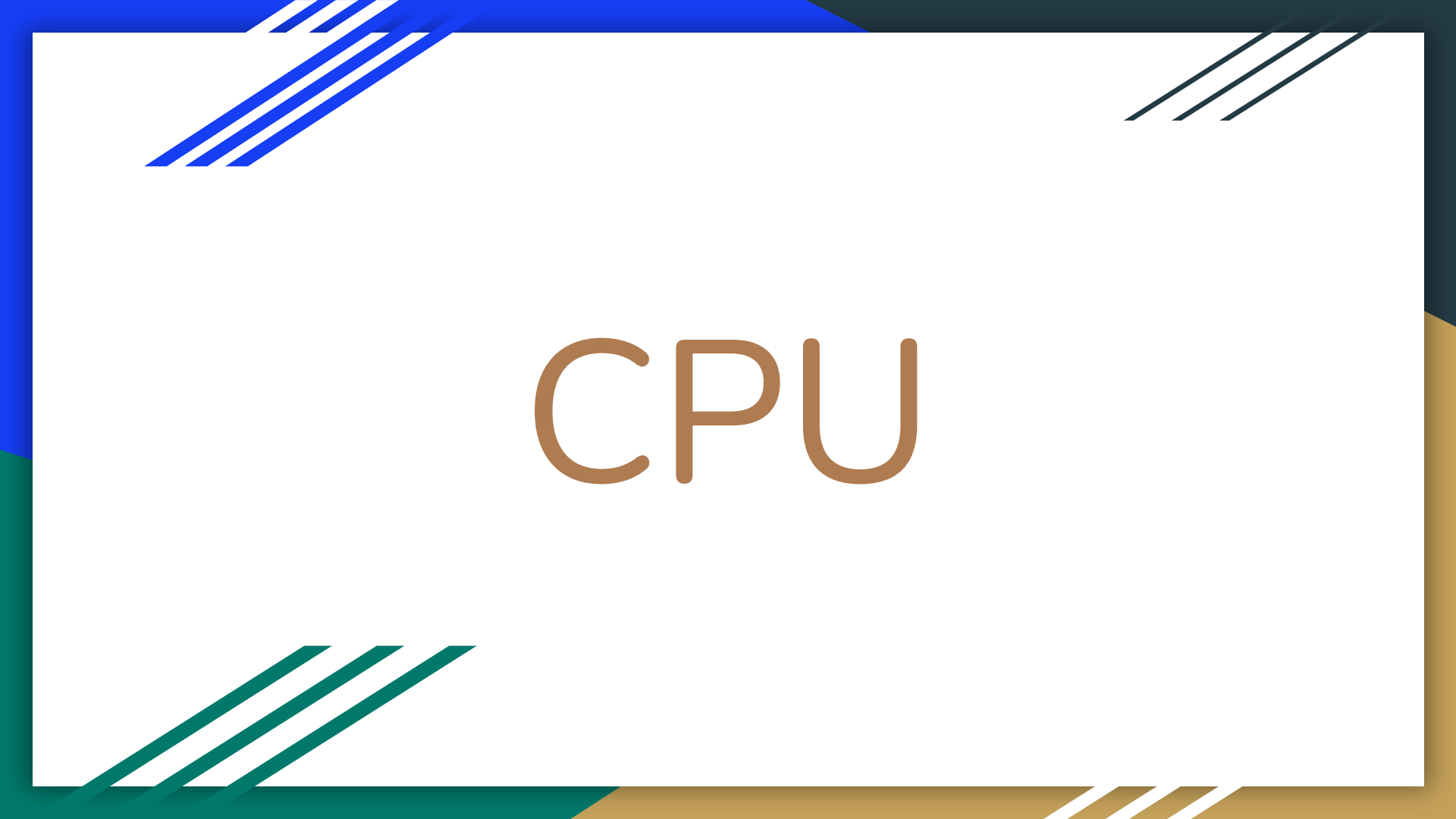


UE Parpaing Galax



LETARD Lilian
TRIPARD Quentin



CPU

Openmp parallelism

Openmp first implementation

- Baseline performance of CPU : 2 fps
- Openmp implementation : 5.5 fps

Optimization and removing of if/else statements

- Remove $i == j$ condition
- Improvement 9 fps

```
#pragma omp parallel for
for (int i = 0; i < n_particles; i++)
{
    for (int j = 0; j < n_particles; j++)
    {
        const float diffx = particles.x[j] - particles.x[i];
        const float diffy = particles.y[j] - particles.y[i];
        const float diffz = particles.z[j] - particles.z[i];
```

if -> fmax

9 fps -> 9.5 fps

Avant

```
if (dij < 1.0)
{
    dij = 10.0;
}
else
{
    dij = std::sqrt(dij);
    dij = 10.0 / (dij * dij * dij);
}
```

Après

```
dij = std::fmax(dij, 1.0);
dij = std::sqrt(dij);
dij = 10.0 / (dij * dij * dij);
```

SIMD Implementation

Simd of the inner loop (j loop) : 60 fps

- Took 8 particles j and one i
- Openmp parallele on i loop
- Usage of mask for computing the distance check

```
float pi_x = particles.x[i];
for (; j < n_particles - simd_size + 1; j += simd_size) {
    auto pj_x = xs::load_unaligned(&particles.x[j]);
    auto dx = pj_x - pi_x;
    auto r2 = xs::fma(dx, dx, xs::fma(dy, dy, dz * dz));
    auto mask = r2 < 1.0f;
    auto r = xs::sqrt(r2);
    auto val_far = 10.0f / (r * r2);
    auto factor = xs::select(mask, batch_array(10.0f), val_far);
    auto m_j = xs::load_unaligned(&initstate.masses[j]);
    factor *= m_j;
    ax_sum = xs::fma(dx, factor, ax_sum);
}
accelerationsx[i] += xs::hadd(ax_sum);
```

Rsqrt and multiplication : 100 fps

```
auto r = xs::rsqrt(r2);
auto val_far = 10.0f * r * r * r;
```

Last batch of improvement

Simd of the inner loop (i loop) : 110 fps

- Took 8 particles i and one j broadcasted
- Openmp parallele on i loop
- Less memory hit miss

Compilation using fastmath flag : 116 fps

```
vrsqrtps %ymm0, %ymm0          # Calculates: r = 1.0 / sqrt(dij)
vmulps   %ymm4, %ymm0, %ymm12 # Calculates: ymm12 = 10.0 * r
vmulps   %ymm0, %ymm0, %ymm0    # Calculates: ymm0 = r * r
vmulps   %ymm0, %ymm12, %ymm12 # Calculates: Final dij = (r * r) * (10.0 * r)
```

Reduce memory access : 117 fps

- Took 16 particles i , Increase j on 8 first, Decrease j on 8 last

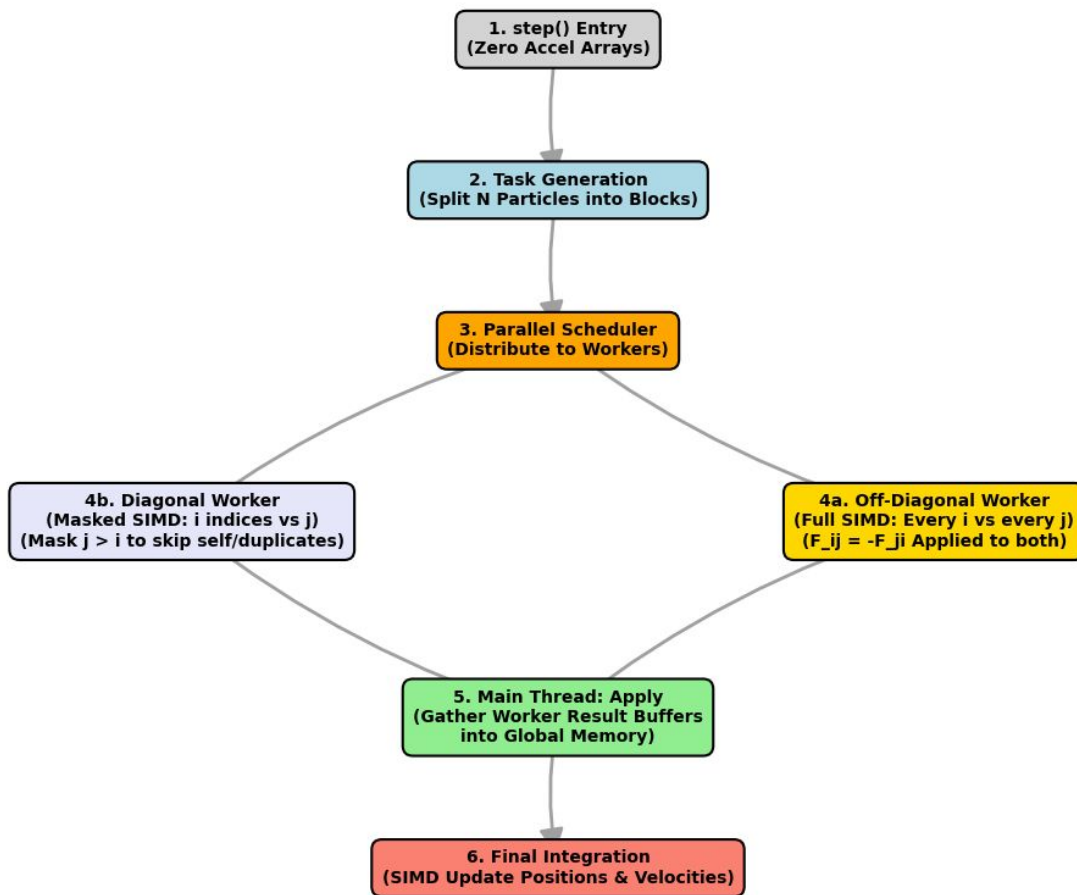
Ablation test :

- Compile forcing avx2 to be off : 30 fps
- Deactivate Openmp : 9.5 fps
- ~ 40 % of the total possible improvement
- Removing load of data in j loop :
 - 230 fps, so a lot of time is wasting loading data

Failed attempt

Use the third law of Newton :

- Concurrent access in memory
- Block computation and only one thread applying result





GPU

Maximum théorique

- GTX 1060 : 4.287 TFLOPS en FP32
- 19 FLOPS / interaction
- $10000 * 10000 * 19 = 1.9 * 10^{**9}$
- $(4.287 * 10^{**12}) / (1.9 * 10^{**9}) = \mathbf{2256 \text{ fps}}$ en calcul pur

Baseline

Portage de CPU naïf en GPU : **55 fps**

Fusion massesGPU et positionsGPU + float4

55 fps -> 100 fps (+81.8%)

Avant

- float3 positionsGPU
- float3 vitessesGPU
- float3 accelerationsGPU
- float massesGPU

Après

- float4 positionsGPU
- float4 vitessesGPU
- float4 accelerationsGPU

chargement depuis la VRAM : 32, 64 ou 128 octets

- float3: 12 octets -> 12, 24, 36, ...
- float4: 16 octets -> 16, 32, 48, 64

Registre local de l'accélération

100 fps -> 350 fps (+250%)

Avant

```
accelerationsGPU[i].x += diffx * dij * positionsGPU[j].w;  
accelerationsGPU[i].y += diffy * dij * positionsGPU[j].w;  
accelerationsGPU[i].z += diffz * dij * positionsGPU[j].w;
```

écriture dans la VRAM

Après

```
    a.x += diffx * dij * positionsGPU[j].w;  
    a.y += diffy * dij * positionsGPU[j].w;  
    a.z += diffz * dij * positionsGPU[j].w;  
}  
accelerationsGPU[i] = a;
```

écriture dans les registres du
GPU + 1 écriture dans la VRAM

if -> fmax

350 fps -> 410 fps (+17.1%)

Avant

```
if (dij < 1.0)
{
    dij = 10.0;
}
else
{
    dij = std::sqrt(dij);
    dij = 10.0 / (dij * dij * dij);
}
```

Après

```
dij = std::sqrt(fmaxf(dij, 1.0f));
dij = 10.0 / (dij * dij * dij);
```

sqrt -> rsqrtf

Avant

```
di_j = std::sqrt(fmaxf(di_j, 1.0f));  
di_j = 10.0 / (di_j * di_j * di_j);
```

Après

```
di_j = rsqrtf(fmaxf(di_j, 1.0f));  
di_j = 10.0 * di_j * di_j * di_j;
```

```
di_j = std::sqrt(fmaxf(di_j, 1.0f));  
di_j = 10.0 / (di_j * di_j * di_j);
```

max.ftz.f32	%f63, %f61, %f62;	<- fmaxf(di_j, 1.0f)
sqrt.approx.ftz.f32	%f64, %f63;	<- std::sqrt()
mul.ftz.f32	%f65, %f64, %f64;	<- di_j * di_j
mul.ftz.f32	%f66, %f64, %f65;	<- (di_j * di_j) * di_j
mov.f32	%f67, 0f41200000;	<- conversion 10.0 -> 10.0f
div.full.ftz.f32	%f68, %f67, %f66;	<- 10.0f / (di_j * di_j * di_j)

sqrt -> rsqrtf

410 fps -> 150 fps (-63.4%)

Avant

```
di_j = std::sqrt(fmaxf(di_j, 1.0f));  
di_j = 10.0 / (di_j * di_j * di_j);
```



Après

```
di_j = rsqrtf(fmaxf(di_j, 1.0f));  
di_j = 10.0 * di_j * di_j * di_j;
```

```

dij = std::sqrt(fmaxf(dij, 1.0f));
dij = 10.0 / (dij * dij * dij);

```

max.ftz.f32	%f63, %f61, %f62;	<- fmaxf(dij, 1.0f)
sqrt.approx.ftz.f32	%f64, %f63;	<- std::sqrt()
mul.ftz.f32	%f65, %f64, %f64;	<- dij * dij
mul.ftz.f32	%f66, %f64, %f65;	<- (dij * dij) * dij
mov.f32	%f67, 0f41200000;	<- conversion 10.0 -> 10.0f
div.full.ftz.f32	%f68, %f67, %f66;	<- 10.0f / (dij * dij * dij)

FP64 (double précision) ?

136.7 GFLOPS

```

dij = rsqrtf(fmaxf(dij, 1.0f));
dij = 10.0 * dij * dij * dij;

```

FP32 (flottant) ?

4.287 TFLOPS

rsqrt.approx.ftz.f32	%f64, %f63;	<- calcul inverse racine
cvt.ftz.f64.f32	%fd1, %f64;	<- conversion 32 -> 64 bits
mul.f64	%fd2, %fd1, 0d4024000000000000;	<- multiplications 64 bits
mul.f64	%fd3, %fd2, %fd1;	<- multiplications 64 bits
mul.f64	%fd4, %fd3, %fd1;	<- multiplications 64 bits
cvt.rn.ftz.f32.f64	%f65, %fd4;	<- conversion 64 -> 32 bits

Solution

```
di_j = rsqrtf(fmaxf(di_j, 1.0f));  
di_j = 10.0f * di_j * di_j * di_j;
```

```
rsqrt.approx.ftz.f32 %f87, %f86;          <- calcul inverse racine  
mul.ftz.f32          %f88, %f87, 0f41200000; <- multiplication 32 bits  
mul.ftz.f32          %f89, %f87, %f88;    <- multiplication 32 bits  
mul.ftz.f32          %f90, %f87, %f89;    <- multiplication 32 bits
```

sqrt -> rsqrtf + 10.0f

410 fps -> 710 fps (+73.1%)

Avant

```
di_j = rsqrtf(fmaxf(di_j, 1.0f));  
di_j = 10.0 * di_j * di_j * di_j;
```

Après

```
di_j = rsqrtf(fmaxf(di_j, 1.0f));  
di_j = 10.0f * di_j * di_j * di_j;
```



Shared memory (tiling)

710 fps -> 1020 fps (+43.6%)

Avant

Lecture des positions dans la
VRAM

-> **10000 trajets VRAM**

Après

Lecture des positions par blocs
dans la mémoire partagée

-> **$10000/512 = 20$ trajets VRAM**

fmaf

1020 fps -> 1050 fps (+2.9%)

$\text{fmaf}(A, B, C) = (A * B) + C$

Avant

```
float dij = diffx * diffx + diffy * diffy + diffz * diffz;
```

Après

```
float temp1 = diffz*diffz;  
float temp2 = fmaf(diffy, diffy, temp1);  
float dij = fmaf(diffx, diffx, temp2);
```

fmaf encore

1050 fps -> 1130 fps (+7.6%)

Avant

```
a.x += diffx * dij * positionsGPU[j].w;  
a.y += diffy * dij * positionsGPU[j].w;  
a.z += diffz * dij * positionsGPU[j].w;
```

Après

```
float jsp = dij * positionsGPU[i].w;  
  
a.x = fmaf(diffx, jsp, a.x);  
a.y = fmaf(diffy, jsp, a.y);  
a.z = fmaf(diffz, jsp, a.z);
```

flag `-use_fast_math`

1130 fps -> 1390 fps (+23.0%)

Avant

`pas -use_fast_math`

Après

`-use_fast_math`

Total

55 fps -> 1390 fps (+2427%) – Erreur max : 10^{-5} (sur 10s)
Erreur moyenne : 10^{-6} (sur 10s)

61.6% du maximum théorique (2256 fps)

Facteur limitants

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start); // Début du chrono GPU
compute_acc_mais_en_mieux<<<nblocks, nthreads>>>(positionsGPU, accelerationsGPU, n_particles);
maj_pos<<<nblocks, nthreads>>>(positionsGPU, velocitiesGPU, accelerationsGPU, n_particles);
cudaEventRecord(stop); // Fin du chrono GPU

cudaEventSynchronize(stop);
float milliseconds = 0;
cudaEventElapsedTime(&milliseconds, start, stop);

printf("FPS GPU PUR %f\n", 1000.0f/milliseconds);

cudaEventDestroy(start);
cudaEventDestroy(stop);
```

FPS GPU PUR 1555.887573d: 1396.96

-> 68,9% du théorique maximal

Facteur limitants

- `__syncthread__`
- chargements mémoire
- boucles
- benchmark
- ...

Merci :)

Progression

- portage de CPU_naif sur GPU : 55 fps
- fusion massesGPU et positionsGPU / float3 -> float4 : 100 fps
- creation d'un float4 accélération temporaire : 350 fps
- if -> fmax : 410 fps
- rsqrtf : 150 fps ;(
- rsqrtf + 10.0f : 710 fps MAIS introduction d'une première erreur
- shared memory : 1020 fps
- fmaf 1 : 1050 fps
- fmaf 2 : 1130 fps
- compilation : use_fast_math dans les cudaflags : 1390 fps